

INSTITUTO FEDERAL
Fluminense

Campus
Bom Jesus do Itabapoana

Algoritmos e estruturas de dados II

Ordenação Interna x externa

Manipulação de arquivos

Professora Ana Mara de Oliveira Figueiredo
ana.figueiredo@iff.edu.br

Ordenação

Ordenar: processo de rearranjar um conjunto de objetos em uma ordem ascendente ou descendente.

- A ordenação visa facilitar a recuperação posterior de itens do conjunto ordenado.
 - Dificuldade de se utilizar um catálogo telefônico se os nomes das pessoas não estivessem listados em ordem alfabética.
- Notação utilizada nos algoritmos:
 - Os algoritmos trabalham sobre os registros de um arquivo.
 - Cada registro possui uma chave utilizada para controlar a ordenação.
 - Podem existir outros componentes em um registro

Ordenação

- Qualquer tipo de chave sobre o qual exista uma regra de ordenação bem definida pode ser utilizado.
- Um método de ordenação é estável se a ordem relativa dos itens com chaves iguais não se altera durante a ordenação.
- Alguns dos métodos de ordenação mais eficientes não são estáveis.
- A estabilidade pode ser forçada quando o método é não-estável.
- Sedgwick (1988) sugere agregar um pequeno índice a cada chave antes de ordenar, ou então aumentar a chave de alguma outra forma.

Ordenação

- Classificação dos métodos de ordenação:
 - Ordenação interna:
 - arquivo a ser ordenado cabe todo na memória principal.
 - qualquer registro pode ser imediatamente acessado.
 - Ordenação externa:
 - arquivo a ser ordenado não cabe na memória principal.
 - os registros são acessados sequencialmente ou em grandes blocos.
- A maioria dos métodos de ordenação é baseada em comparações das chaves.
- Existem métodos de ordenação que utilizam o princípio da distribuição.

Ordenação externa

- A ordenação externa consiste em ordenar arquivos de tamanho maior que a memória interna disponível.
- Os métodos de ordenação externa são muito diferentes dos de ordenação interna.
 - Na ordenação externa os algoritmos devem diminuir o número de acesso às unidades de memória externa.

Ordenação externa

- Nas memórias externas, os dados são armazenados como um arquivo sequencial.
 - Apenas um registro pode ser acessado em um dado momento.
 - Esta é uma restrição forte se comparada com as possibilidades de acesso em um vetor.
- Logo, os métodos de ordenação interna são inadequados para ordenação externa. Técnicas de ordenação completamente diferentes devem ser utilizadas

Ordenação externa

Fatores que determinam as diferenças das técnicas de ordenação externa:

1. Custo para acessar um item é algumas ordens de grandeza maior.
2. O custo principal na ordenação externa é relacionado a transferência de dados entre a memória interna e externa.
3. Existem restrições severas de acesso aos dados.
4. O desenvolvimento de métodos de ordenação externa é muito dependente do estado atual da tecnologia.
5. A variedade de tipos de unidades de memória externa torna os métodos dependentes de vários parâmetros.

Ordenação externa

- O método mais importante é o de ordenação por intercalação.
- Intercalar significa combinar dois ou mais blocos ordenados em um único bloco ordenado.
- A intercalação é utilizada como uma operação auxiliar na ordenação.

Ordenação externa

- Os algoritmos para ordenação externa devem reduzir o número de passadas sobre o arquivo.
- Uma boa medida de complexidade de um algoritmo de ordenação por intercalação é o número de vezes que um item é lido ou escrito na memória auxiliar.
- Os bons métodos de ordenação geralmente envolvem no total menos do que dez passadas sobre o arquivo.

Estratégia geral dos métodos de ordenação externa:

1. Quebre o arquivo em blocos do tamanho da memória interna disponível.
2. Ordene cada bloco na memória interna.
3. Intercale os blocos ordenados, fazendo várias passadas sobre o arquivo.
4. A cada passada são criados blocos ordenados cada vez maiores, até que todo o arquivo esteja ordenado.

Manipulação de arquivos

bibliotecas: `stdio.h` em C e `cstdio` em C++

`fopen` - Abre um arquivo para operações de leitura ou escrita.

```
FILE *fopen(const char *filename, const char *mode);
```

`fclose` - Fecha um arquivo previamente aberto.

```
int fclose(FILE *stream);
```

`fprintf` - Escreve dados em um arquivo formatado.

```
int fprintf(FILE *stream, const char *format, ...);
```

fscanf - Lê dados formatados de um arquivo.

```
int fscanf(FILE *stream, const char *format, ...);
```

fread - Lê blocos de dados de um arquivo.

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

fwrite - Escreve blocos de dados em um arquivo.

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
```

fseek - Define a posição do ponteiro do arquivo.

```
int fseek(FILE *stream, long int offset, int whence);
```

ftell - Retorna a posição atual do ponteiro do arquivo.

```
long int ftell(FILE *stream);
```

rewind - Move o ponteiro do arquivo para o início do arquivo.

```
void rewind(FILE *stream);
```

remove - Remove um arquivo.

```
int remove(const char *filename);
```

rename - Renomeia um arquivo.

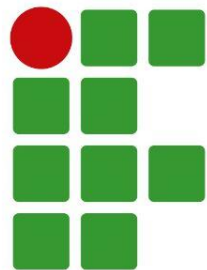
```
int rename(const char *old_filename, const char *new_filename);
```

```

1  #include <stdio>
2  using namespace std;
3
4  int main() {
5      FILE *arquivo_saida = fopen("exemplo.txt", "w");
6      if (arquivo_saida != nullptr) {
7          fprintf(arquivo_saida, "Olá, mundo!\n");
8          fprintf(arquivo_saida, "Esta é uma linha escrita no arquivo.\n");
9          fclose(arquivo_saida);
10     } else {
11         fprintf(stderr, "Não foi possível abrir o arquivo para escrita.\n");
12         return 1;
13     }
14     FILE *arquivo_entrada = fopen("exemplo.txt", "r");
15     if (arquivo_entrada != nullptr) {
16         char linha[100];
17         //Note que %99[^\n]\n no fscanf é usado para ler linhas do arquivo (evitando estouro de buffer)
18         while (fscanf(arquivo_entrada, "%99[^\n]\n", linha) != EOF) {
19             printf("%s\n", linha);
20         }
21         fclose(arquivo_entrada);
22     } else {
23         fprintf(stderr, "Não foi possível abrir o arquivo para leitura.\n");
24         return 1;
25     }
26     return 0;
27 }
28

```

```
1 #include <iostream>
2 #include <fstream> // Esta biblioteca é derivada da stdio.h
3 using namespace std;
4 int main() {
5     ofstream arquivo_saida("exemplo.txt");
6
7     if (arquivo_saida.is_open()) {
8         arquivo_saida << "Olá, mundo!" << endl;
9         arquivo_saida << "Esta é uma linha escrita no arquivo." << endl;
10        arquivo_saida.close();
11    } else {
12        cerr << "Não foi possível abrir o arquivo." << endl;
13        return 1;
14    }
15
16    ifstream arquivo_entrada("exemplo.txt");
17
18    if (arquivo_entrada.is_open()) {
19        string linha;
20        while (getline(arquivo_entrada, linha)) {
21            cout << linha << endl;
22        }
23        arquivo_entrada.close();
24    } else {
25        cerr << "Não foi possível abrir o arquivo para leitura." << endl;
26        return 1;
27    }
28
29    return 0;
30 }
31
```

INSTITUTO FEDERAL
Fluminense

Campus
Bom Jesus do Itabapoana

Estrutura de dados II

Ordenação externa:

Intercalação Balanceada de Vários Caminhos

Professora Ana Mara de Oliveira Figueiredo
ana.figueiredo@iff.edu.br

Intercalação Balanceada de Vários Caminhos

Considere um arquivo armazenado em uma fita de entrada:

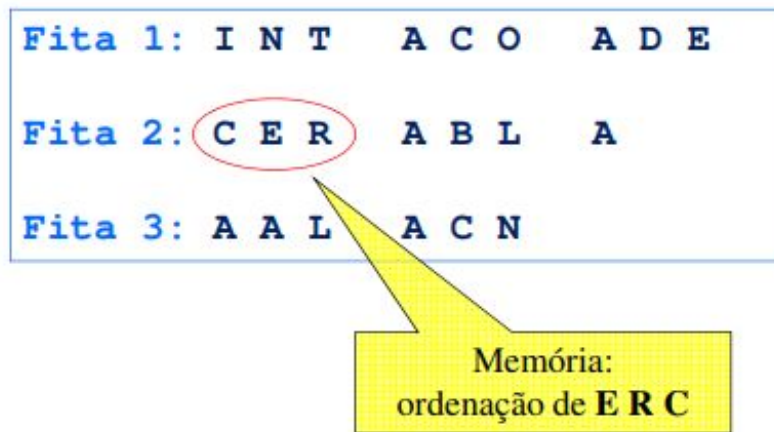
I N T E R C A L A C A O B A L A N C E A D A

Objetivo: – Ordenar os 22 registros e colocá-los em uma fita de saída.

- Os registros são lidos um após o outro.
- Considere uma memória interna com capacidade para três registros.
- Considere que estejam disponíveis seis unidades de fita magnética.

- Fase de criação dos blocos ordenados, envolvendo:
 - quebra do arquivo em blocos do tamanho da memória interna disponível;
 - ordenação de cada bloco na memória interna.

INTERCALACAO BALANCEADA



Fase de intercalação, envolvendo os passos:

- 1. Leitura do primeiro registro de cada fita.
- 2. Retirada do registro contendo a menor chave, armazenando-o em uma fita de saída.
- 3. Leitura de um novo registro da fita de onde o registro retirado é proveniente.
 - Ao ler o terceiro registro de um dos blocos, a fita correspondente fica inativa.
 - A fita é reativada quando o terceiro registro das outras fitas forem lidos.
 - Neste momento, um bloco de nove registros ordenados foi formado na fita de saída.
- 4. Repetição do processo para os blocos restantes.

INTERCALACAO BALANCEADA

Resultado da 1ª passada sobre o arquivo na fase de intercalação:

```
Fita 4: A A C E I L N R T  
Fita 5: A A A B C C L N O  
Fita 6: A A D E
```

Repetindo o processo de intercalação nos blocos formados após a 1ª passada sobre o arquivo, tem-se o arquivo ordenado (2ª passada sobre o mesmo):

```
Fita 1: A A A A A A A B C C C D E E I L L N N O R T  
Fita 2:  
Fita 3:
```

Quantas passadas são necessárias para ordenar um arquivo de tamanho arbitrário?

- Seja n , o número de registros do arquivo.
- Suponha que cada registro ocupa m palavras na memória interna, sendo uma palavra igual a um registro do arquivo.
- A primeira etapa produz n/m blocos ordenados.
- Seja $P(n)$ o número de passadas para a fase de intercalação.
- Seja f o número de fitas utilizadas em cada passada.
- Assim: **$P(n) = \log f(n/m)$** .

No exemplo, $n=22$, $m=3$ e $f=3$ temos: $P(n) = \log (3 * (22/3)) \approx 2$.

- No exemplo foram utilizadas $2f$ fitas para uma intercalação-de-f-caminhos.
- É possível usar apenas $f + 1$ fitas:
 - Encaminhe todos os blocos para uma única fita.
 - Redistribuir estes blocos entre as fitas de onde eles foram lidos.
 - O custo envolvido é uma passada a mais em cada intercalação.
- No caso do exemplo de 22 registros, apenas quatro fitas seriam suficientes:
 - A intercalação dos blocos a partir das fitas 1, 2 e 3 seria toda dirigida para a fita 4.
 - Ao final, o segundo e o terceiro blocos ordenados de nove registros seriam transferidos de volta para as fitas 1 e 2.

- Arquivo original

5	28	10	40	35	7	12	2	21	11	29	27	9	38	8	49	3	15	13	30
17	46	18	36	1	4	34	16	19	22	20									

Etapa 1: Distribuição

Os registros serão lidos em blocos de grande tamanho (4 neste exemplo) para a memória principal, ordenados em memória principal (Quicksort, Mergesort, ...) e escritos nos primeiros M caminhos (2 neste exemplo)

- Arquivo original

5	28	10	40	35	7	12	2	21	11	29	27	9	38	8	49	3	15	13	30
17	46	18	36	1	4	34	16	19	22	20									

- Distribuição

5	10	28	40
---	----	----	----

Ordenação em memória principal

Arq.Temp.1:

5	10	28	40
---	----	----	----

Escrita no primeiro destino

Arq.Temp.2:

- Arquivo original

5	28	10	40	35	7	12	2	1	11	29	27	9	38	8	49	3	15	13	30
17	46	18	36	1	4	34	16	19	22	20									

- Distribuição

2	7	12	35
---	---	----	----

Ordenação em memória principal

Arq.Temp.1:

5	10	28	40
---	----	----	----

Arq.Temp.2:

2	7	12	35
---	---	----	----

 Escrita no segundo destino

- Arquivo original

5	28	10	40	35	7	12	2	21	11	29	27	9	38	8	49	3	15	13	30
17	46	18	36	1	4	34	16	19	22	20									

- Distribuição

Arq.Temp.1:

5	10	28	40	11	21	27	29
---	----	----	----	----	----	----	----

Arq.Temp.2:

2	7	12	35
---	---	----	----

Ordenação em memória principal e escrita no primeiro destino (todos os destinos já receberam um segmento ordenado)

- Arquivo original

5	28	10	40	35	7	12	2	21	11	29	27	9	38	8	49	3	15	13	30
17	46	18	36	1	4	34	16	19	22	20									

- Distribuição

Arq.Temp.1:	5	10	28	40	11	21	27	29	3	13	15	30	1	4	16	34
Arq.Temp.2:	2	7	12	35	8	9	38	49	17	18	36	46	19	20	22	

- Arquivo original

5	28	10	40	35	7	12	2	21	11	29	27	9	38	8	49	3	15	13	30
17	46	18	36	1	4	34	16	19	22	20									

- Distribuição

Arq.Temp.1:	5	10	28	40	11	21	27	29	3	13	15	30	1	4	16	34
Arq.Temp.2:	2	7	12	35	8	9	38	49	17	18	36	46	19	20	22	

- Primeira intercalação

Fontes (fluxos de entrada)

Arq.Temp.1:

5	10	28	40	11	21	27	29	3	13	15	30	1	4	16	34
---	----	----	----	----	----	----	----	---	----	----	----	---	---	----	----

Arq.Temp.2:

2	7	12	35	8	9	38	49	17	18	36	46	19	20	22
---	---	----	----	---	---	----	----	----	----	----	----	----	----	----

Arq.Temp.3:

Destinos (fluxos de saída)

Arq.Temp.4:

- Primeira intercalação

Arq.Temp.1:

5	10	28	40	11	21	27	29	3	13	15	30	1	4	16	34
---	----	----	----	----	----	----	----	---	----	----	----	---	---	----	----

Arq.Temp.2:

2	7	12	35	8	9	38	49	17	18	36	46	19	20	22
---	---	----	----	---	---	----	----	----	----	----	----	----	----	----

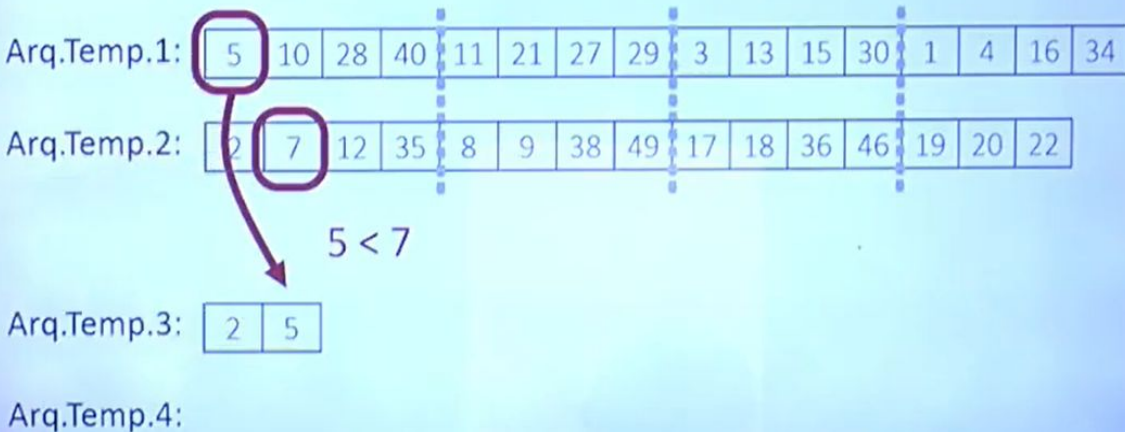
$2 < 5$

Arq.Temp.3:

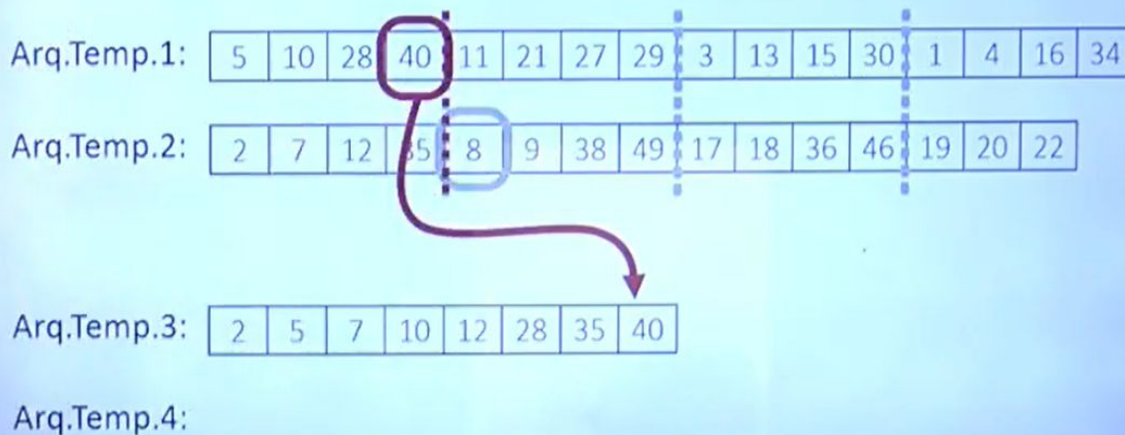
2

Arq.Temp.4:

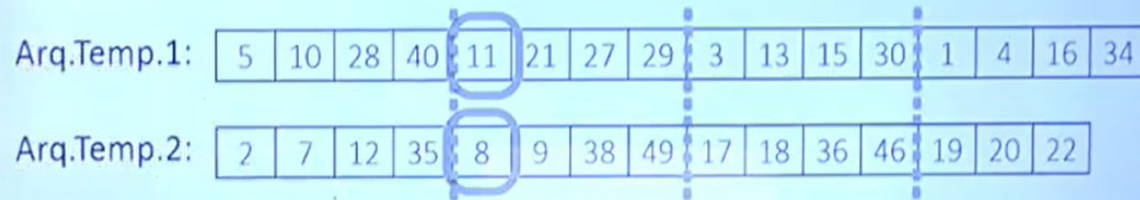
- Primeira intercalação



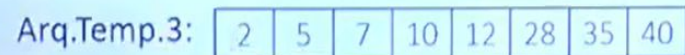
- Primeira intercalação



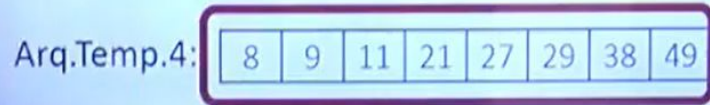
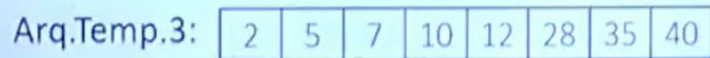
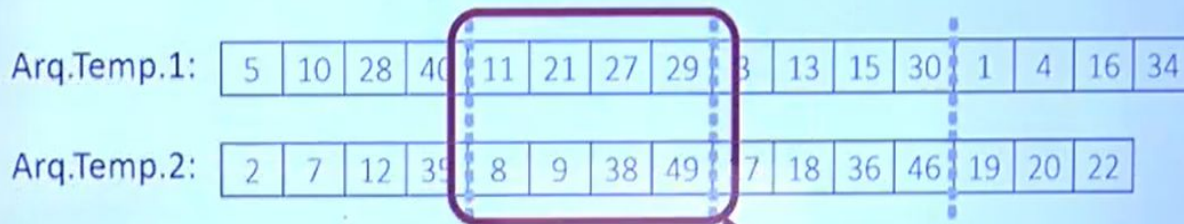
- Primeira intercalação



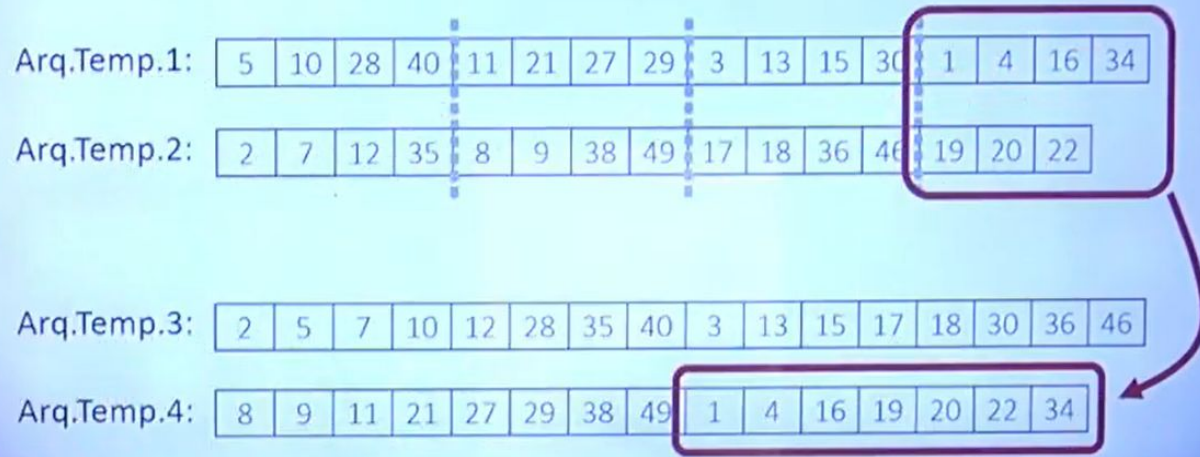
A intercalação de 2 segmentos ordenados de tamanho 4 gera um segmento ordenado de tamanho 8



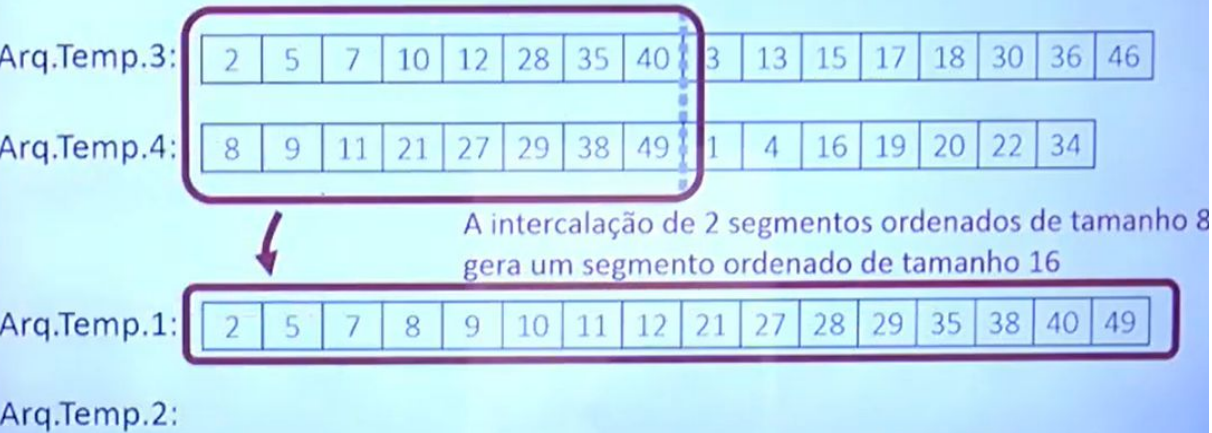
- Primeira intercalação



• Primeira intercalação



• Segunda intercalação



- Segunda intercalação

Arq.Temp.3:	2	5	7	10	12	28	35	40	3	13	15	17	18	30	36	46
Arq.Temp.4:	8	9	11	21	27	29	38	49	1	4	16	19	20	22	34	

Arq.Temp.1:	2	5	7	8	9	10	11	12	21	27	28	29	35	38	40	49
Arq.Temp.2:	1	3	4	13	15	16	17	18	19	20	22	30	34	36	46	

- Terceira intercalação

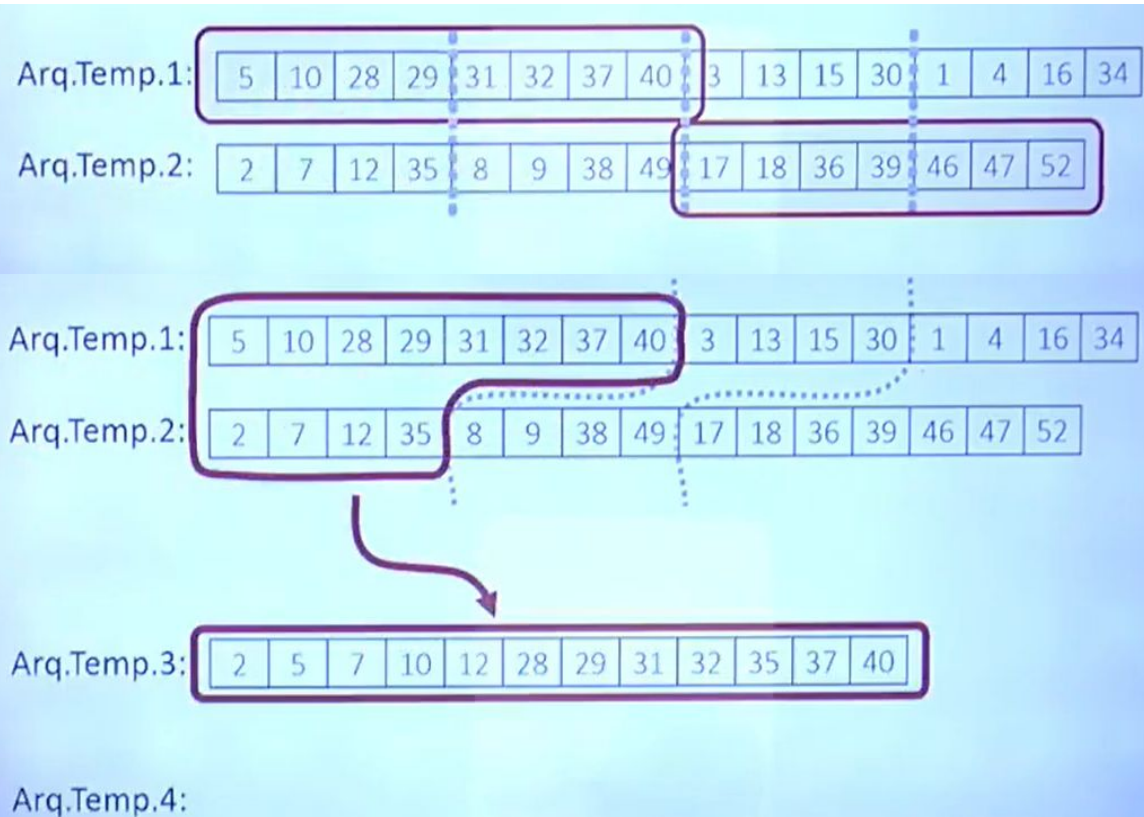
Arq.Temp.1:	2	5	7	8	9	10	11	12	21	27	28	29	35	38	40	49
Arq.Temp.2:	1	3	4	13	15	16	17	18	19	20	22	30	34	36	46	

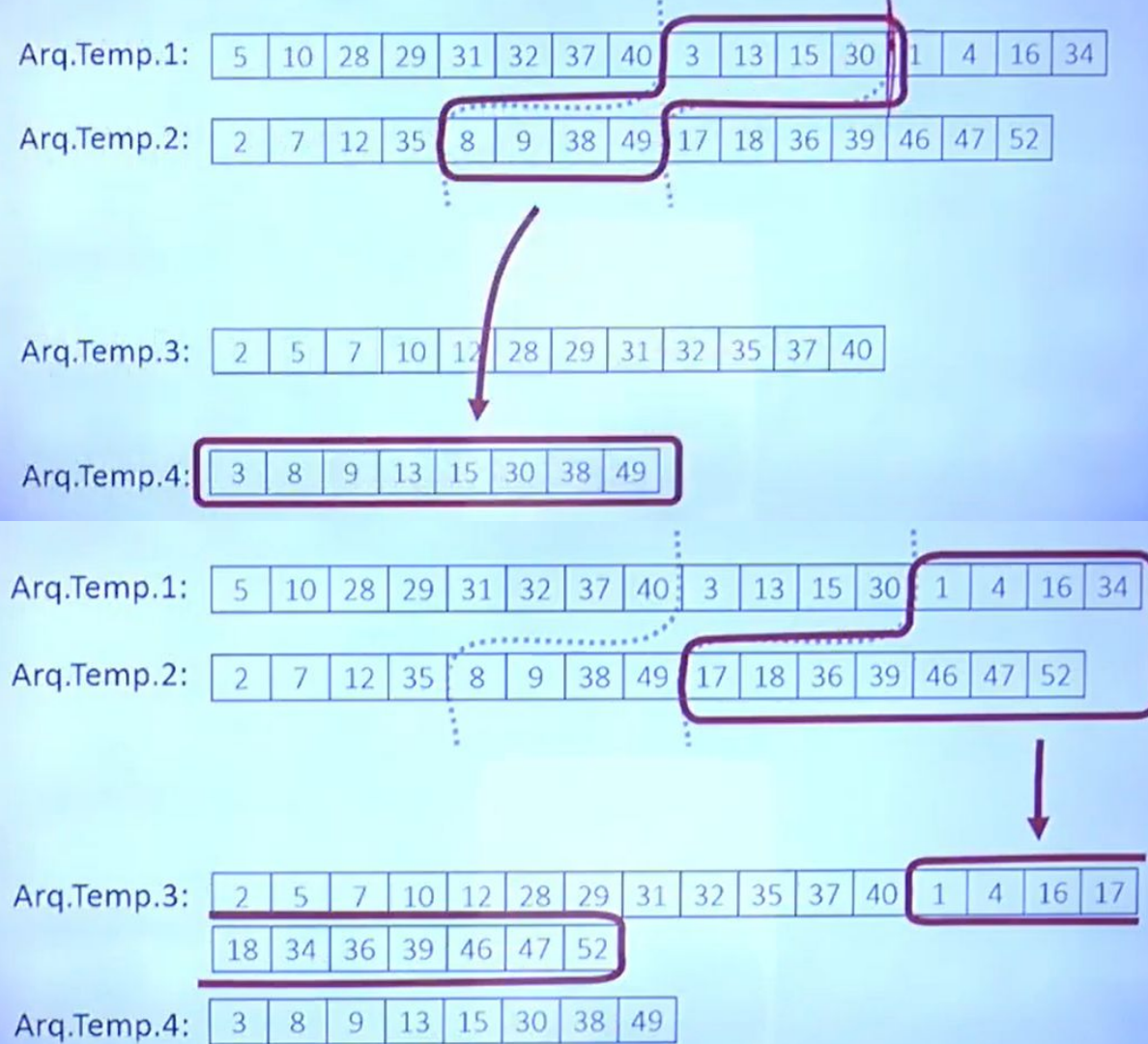
A intercalação de 2 segmentos ordenados de tamanho 16 gera um segmento ordenado de tamanho 32

Arq.Temp.3:	1	2	3	4	5	7	8	9	10	11	12	13	15	16	17	18
	19	20	21	22	27	28	29	30	34	35	36	38	40	46	49	

Arq.Temp.4:

Segmentos de tamanho variável





Arq.Temp.3:

2	5	7	10	12	28	29	31	32	35	37	40	1	4	16	17
18	34	36	39	46	47	52									

Arq.Temp.4:

3	8	9	13	15	30	38	49
---	---	---	----	----	----	----	----

Arq.Temp.1:

2	3	5	7	8	9	10	12	13	15	28	29	30	31	32	35
37	38	40	49												

Arq.Temp.2:

Arq.Temp.1:

2	3	5	7	8	9	10	12	13	15	28	29	30	31	32	35
37	38	40	49												

Arq.Temp.2:

1	4	16	17	18	34	36	39	46	47	52
---	---	----	----	----	----	----	----	----	----	----

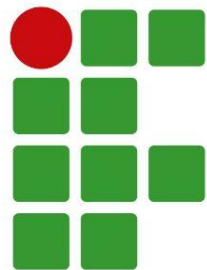
Arq.Temp.3:

1	2	3	4	5	7	8	9	10	12	13	15	16	17	18	28
29	30	31	32	34	35	36	37	38	39	40	46	47	49	52	

Arq.Temp.4:

Problemas com a intercalação balanceada de vários caminhos:

- Necessita de um grande número de fitas, fazendo várias leituras e escritas entre as fitas envolvidas.
 - Para uma intercalação balanceada de f caminhos são necessárias, geralmente, $2f$ fitas.
 - Alternativamente, pode-se copiar o arquivo de uma única fita de saída para f fitas de entrada, reduzindo o número de fitas para $f + 1$.
- Há um custo de uma cópia adicional do arquivo.



INSTITUTO FEDERAL
Fluminense

Campus
Bom Jesus do Itabapoana

Estrutura de dados II

Ordenação externa: Intercalação Polifásica

Professora Ana Mara de Oliveira Figueiredo
ana.figueiredo@iff.edu.br

Intercalação polifásica

Processo de funcionamento da intercalação polifásica:

- Os blocos ordenados são distribuídos de forma desigual entre as fitas disponíveis.
- Uma fita é deixada livre.
- Em seguida, a intercalação de blocos ordenados é executada até que uma das fitas de entrada se esvazie.
- A fita vazia torna-se a próxima fita de saída.

Intercalação com seleção por substituição

Usa fila de prioridades

Heap de mínimo (filhos são maiores que pai)

filho esquerdo: $2i+1$

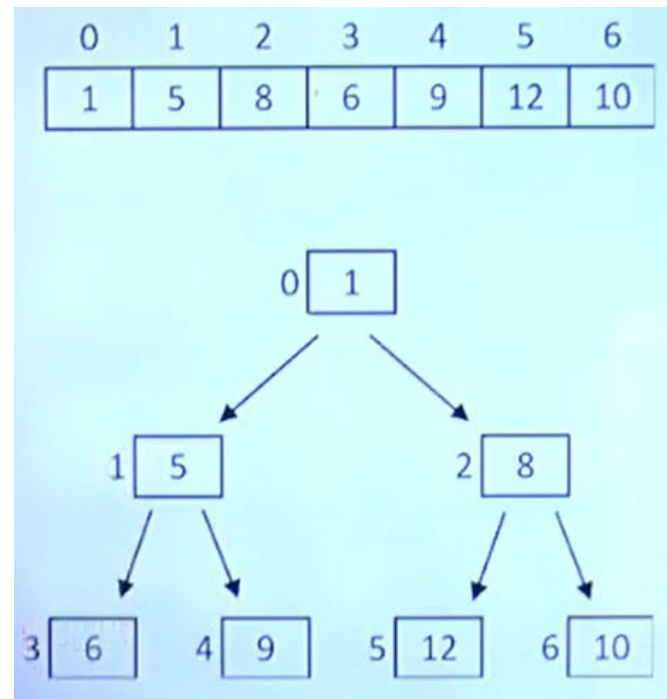
filho direito: $2i+2$

pai: $\text{int}((i-1)/2)$

operações:

inserir na raiz e jogar p/ baixo caso fira alguma regra

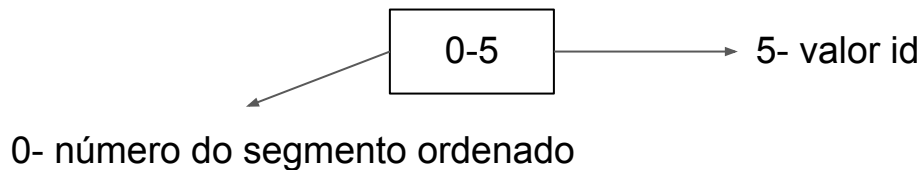
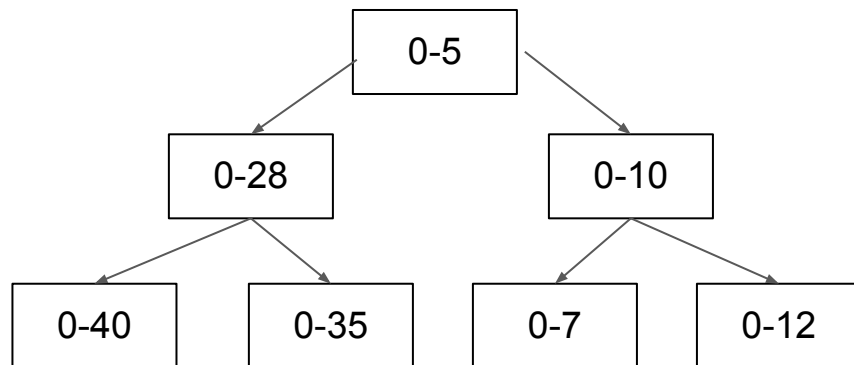
remover a raiz e colocar o novo valor no lugar



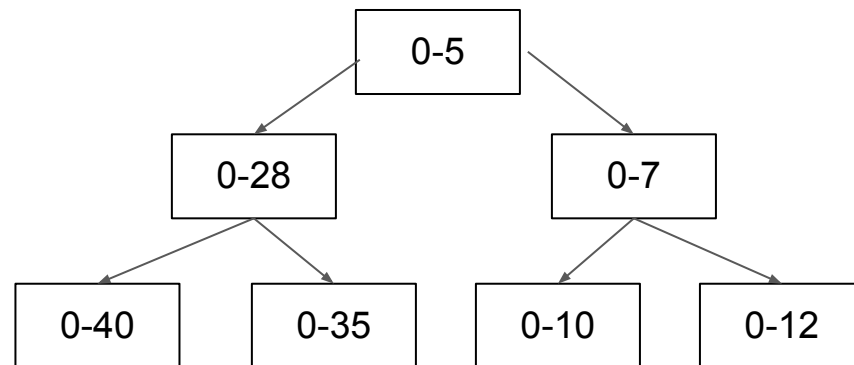
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

1- Preencher o heap

0	1	2	3	4	5	6
0-5	0-28	0-10	0-40	0-35	0-7	0-12



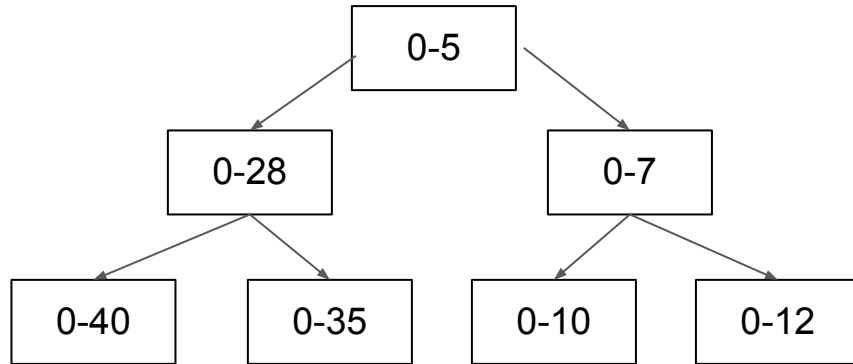
2- organizar o heap



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

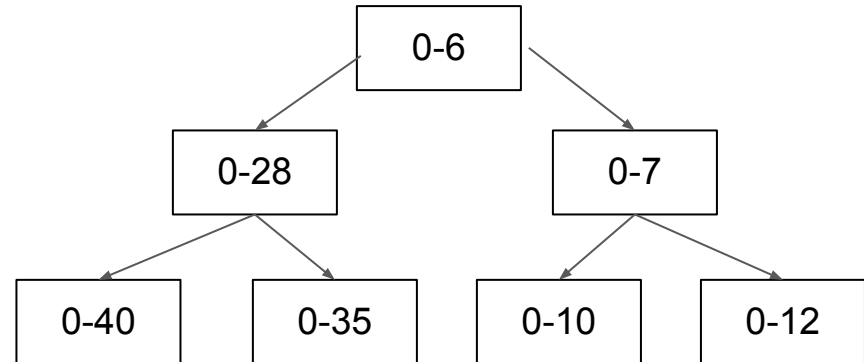
e insere próximo registro na raiz



saídas:

Arq1: 0-5;

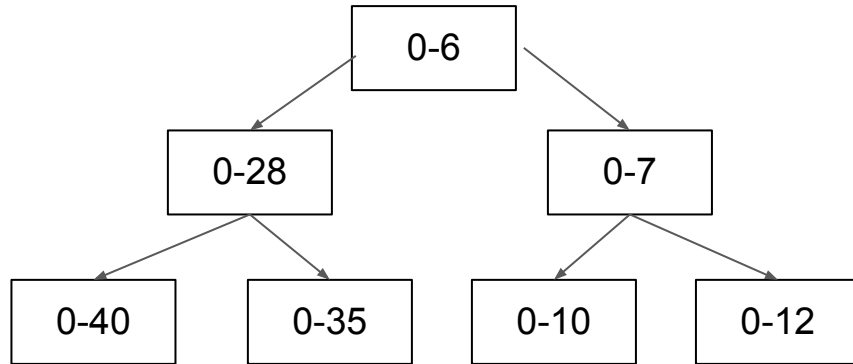
Arq2:



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

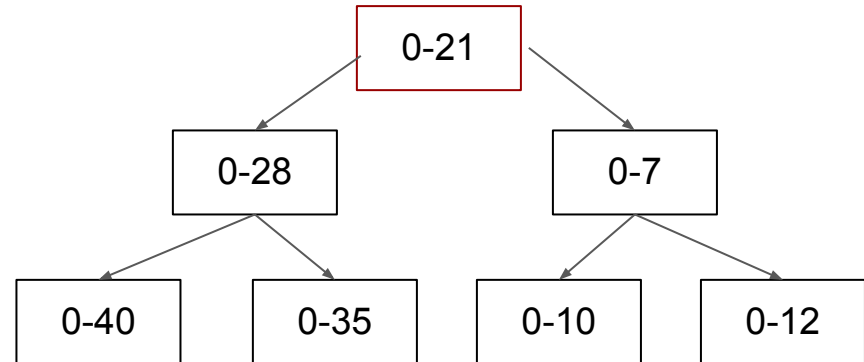
e insere próximo registro na raiz



saídas:

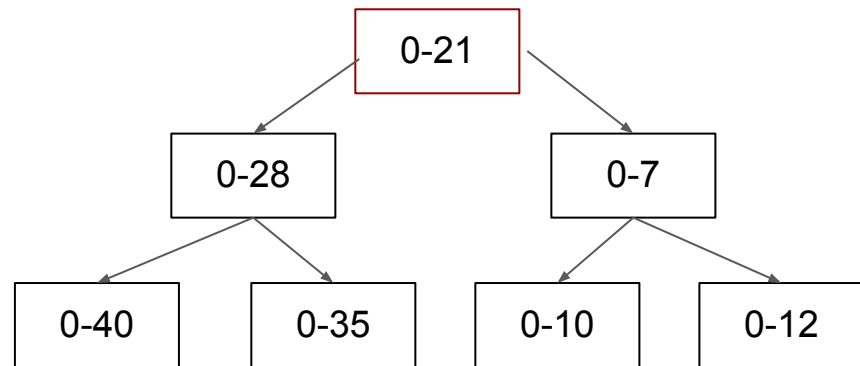
Arq1: 0-5;0-6

Arq2:



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

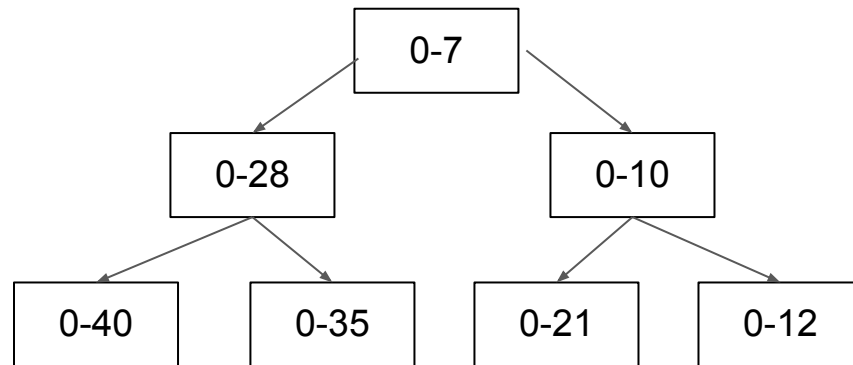
organiza o heap



saídas:

Arq1: 0-5;0-6

Arq2:

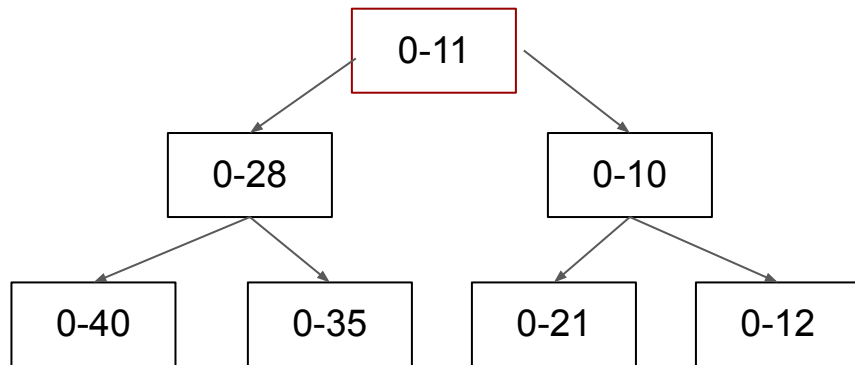


0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

e insere próximo registro na raiz

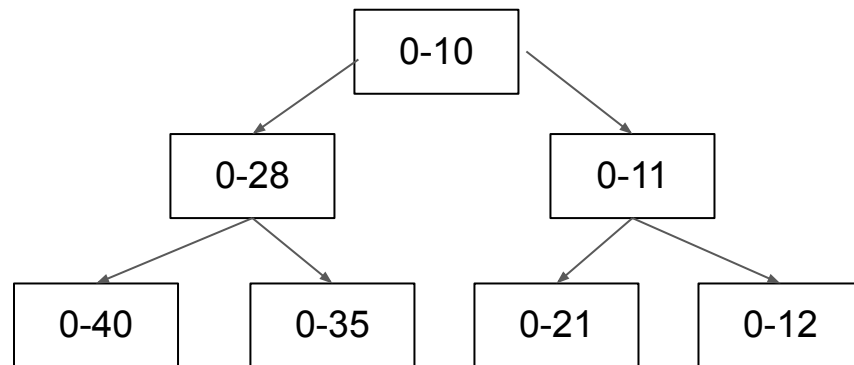
organiza a heap



saídas:

Arq1: 0-5; 0-6; 0-7;

Arq2:



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

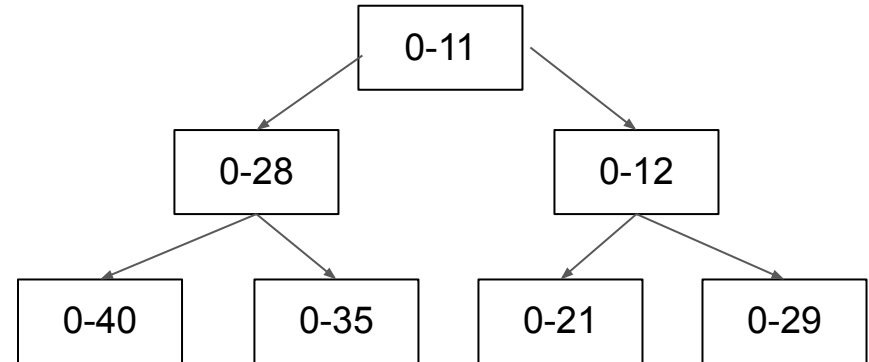
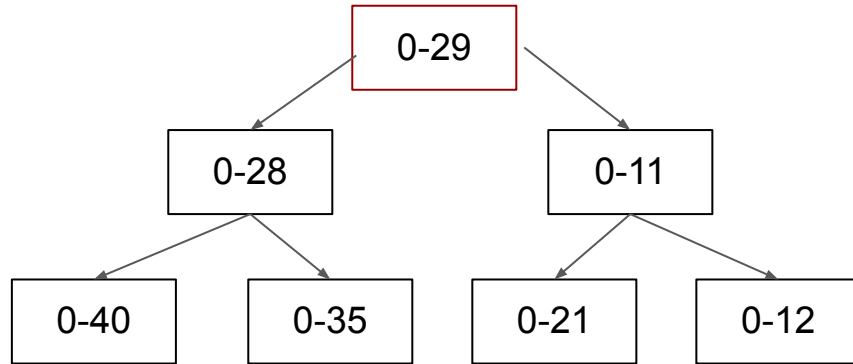
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10;

Arq2:

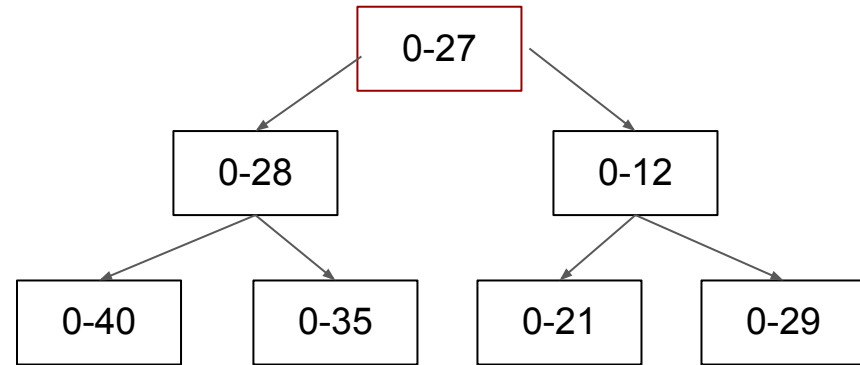


0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

e insere próximo registro na raiz

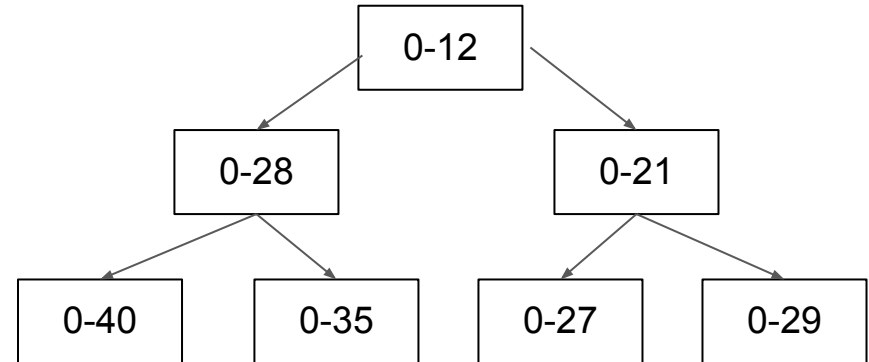
organiza a heap



saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11;

Arq2:

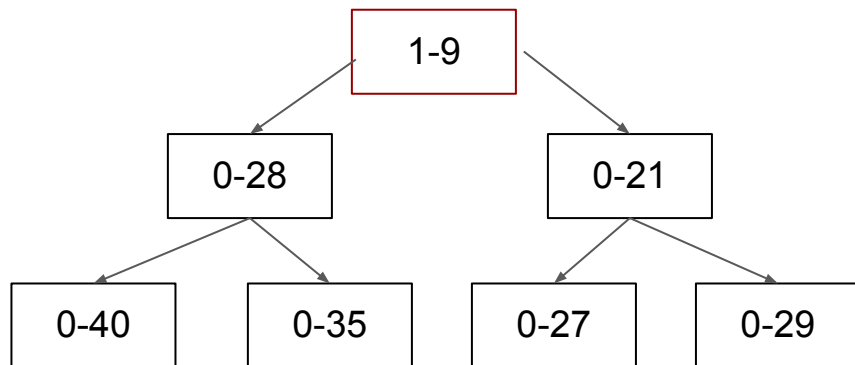


0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

e insere próximo registro na raiz

organiza a heap

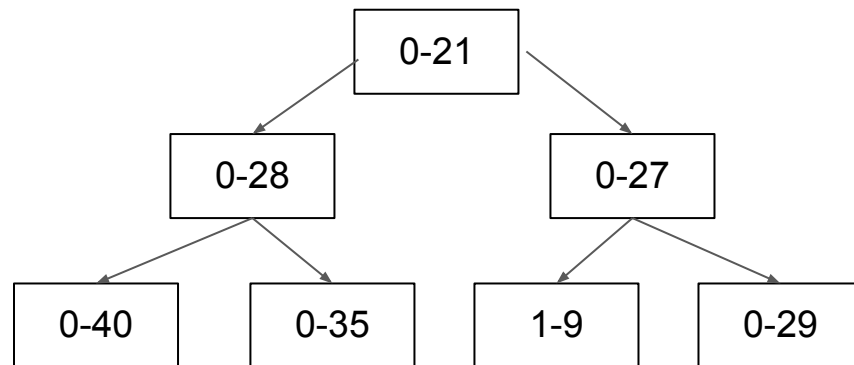


saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12;

Arq2:

9 < 12
se for colocado aqui ele
interromperia o segmento
ordenado

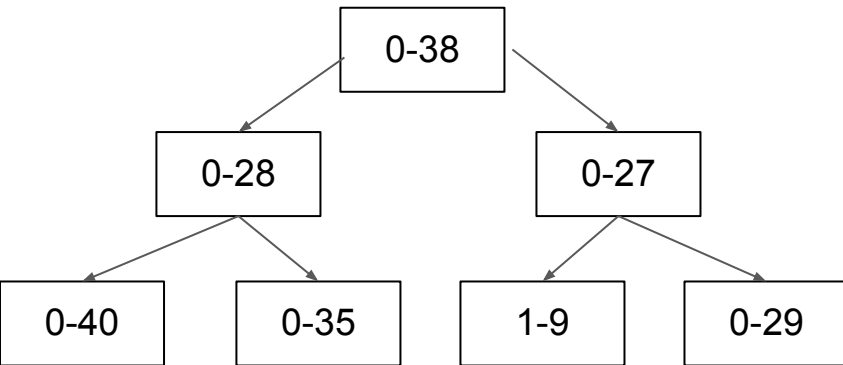


0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

e insere próximo registro na raiz

organiza a heap

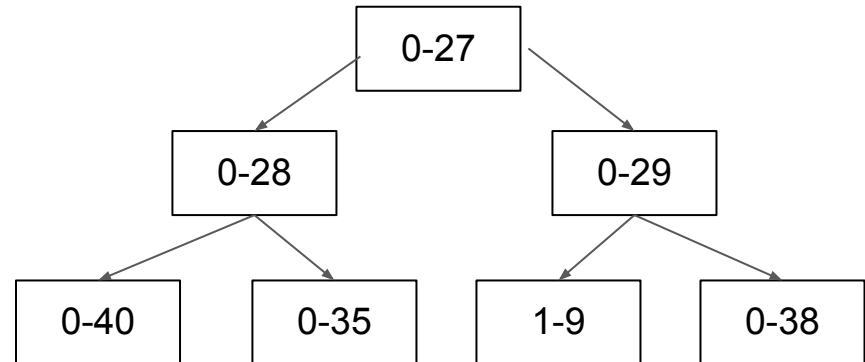


saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21;

Arq2:

38 < 21
pode entrar nesse segmento

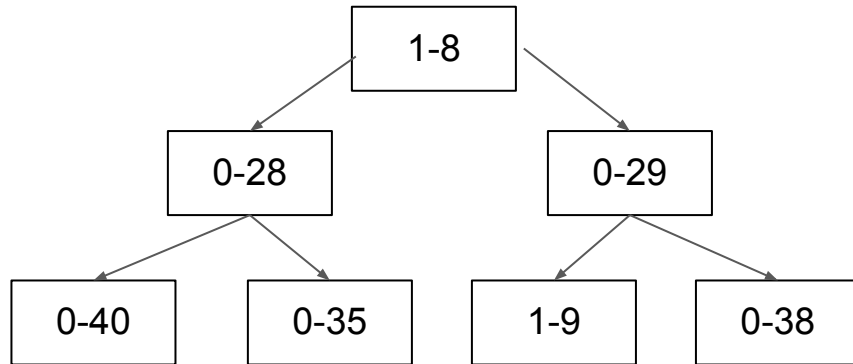


0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

e insere próximo registro na raiz

organiza a heap

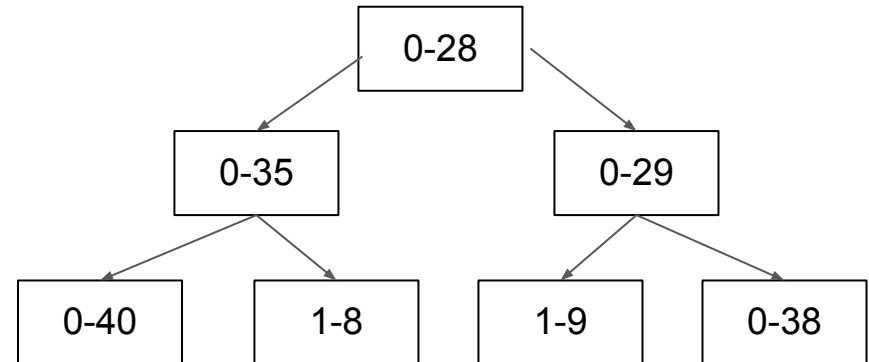


saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27;

Arq2:

8 < 27
se for colocado aqui ele
interromperia o segmento
ordenado

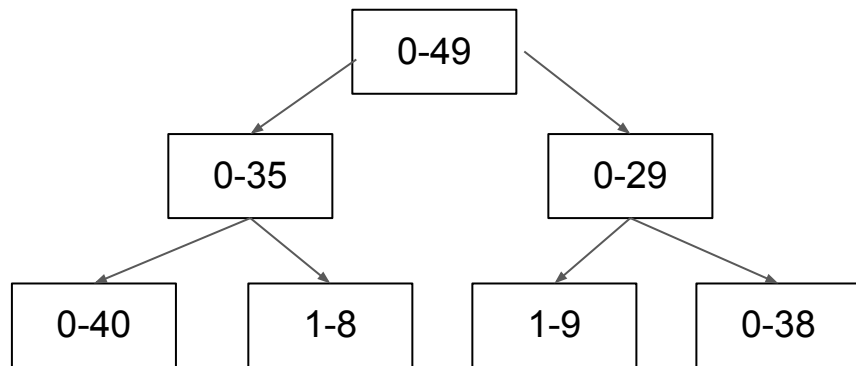


0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

e insere próximo registro na raiz

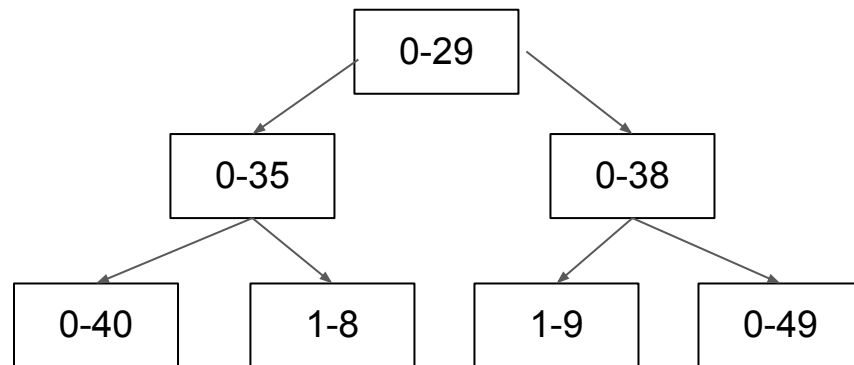
organiza a heap



saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28

Arq2:



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

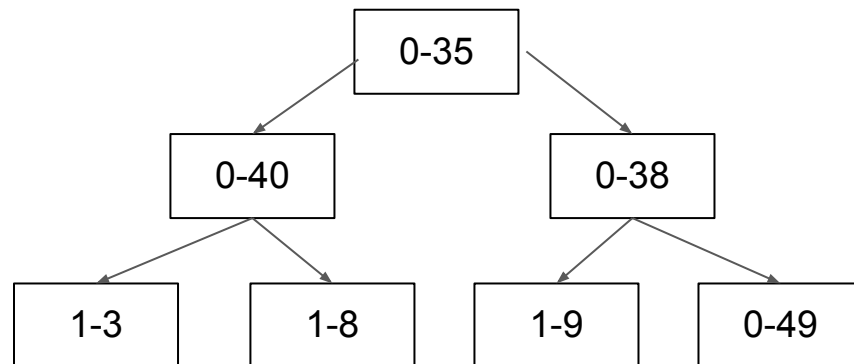
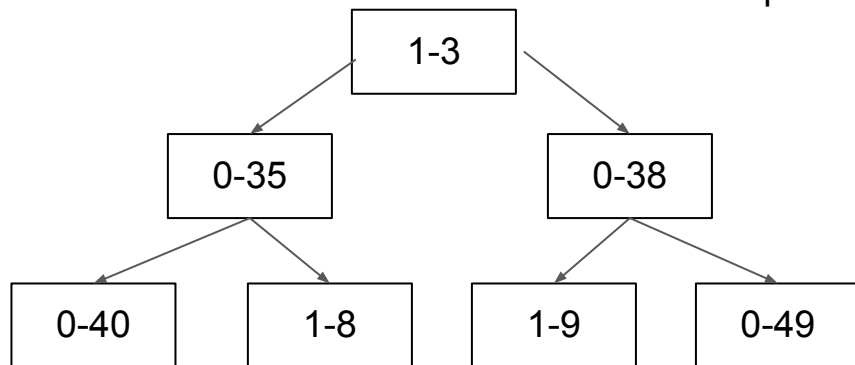
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29;

Arq2:



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

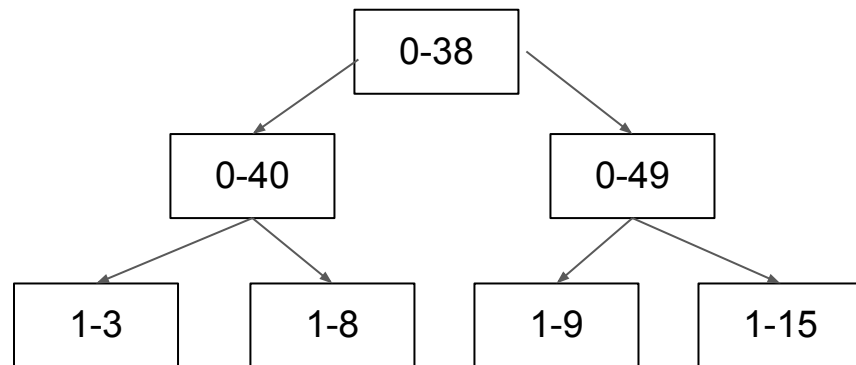
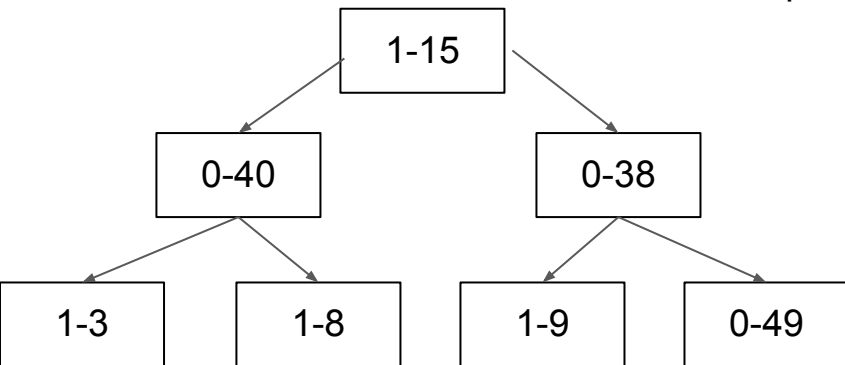
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;

Arq2:



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

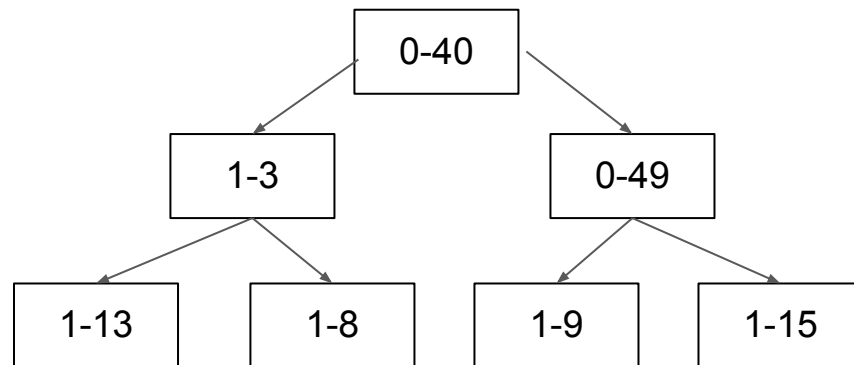
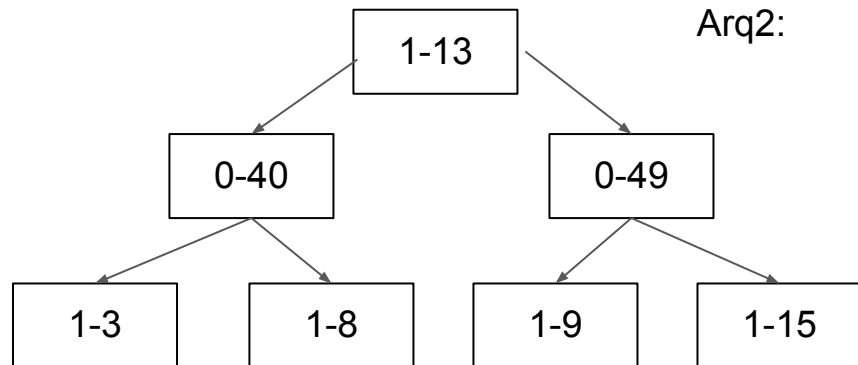
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38;

Arq2:



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

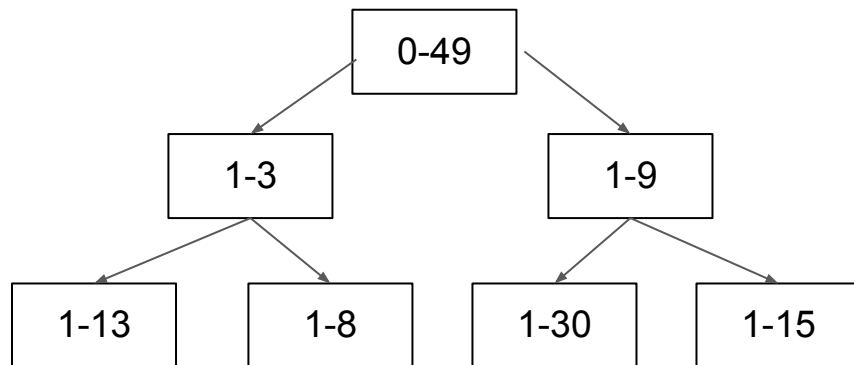
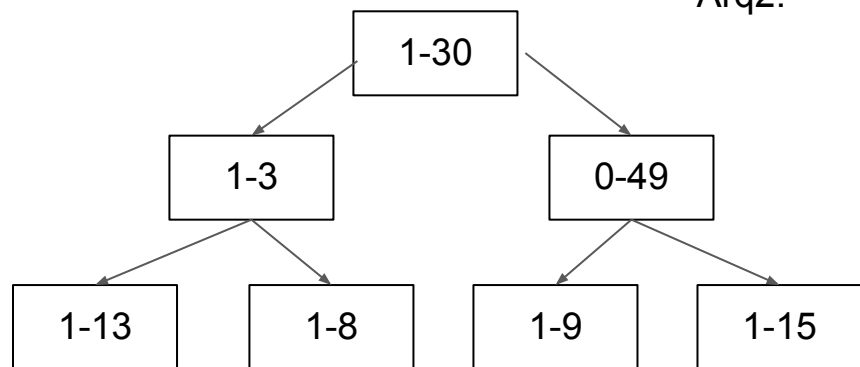
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40;

Arq2:



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

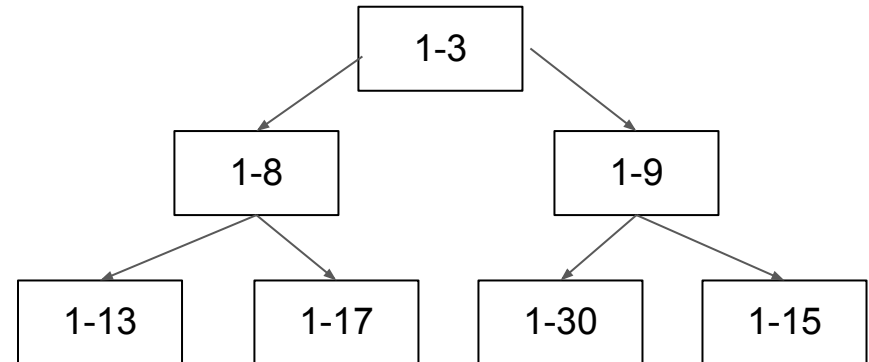
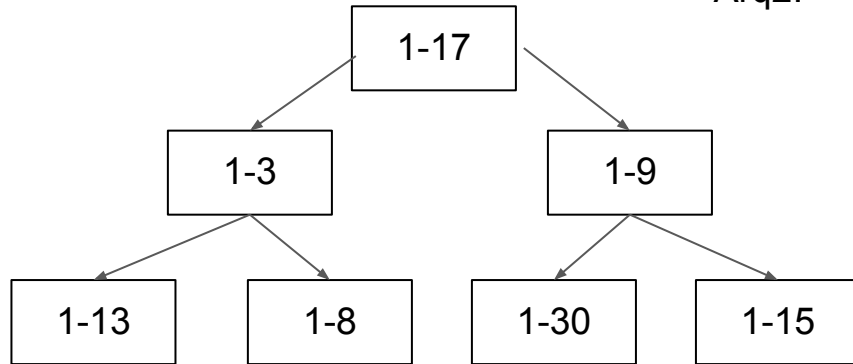
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2:



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

e insere próximo registro na raiz

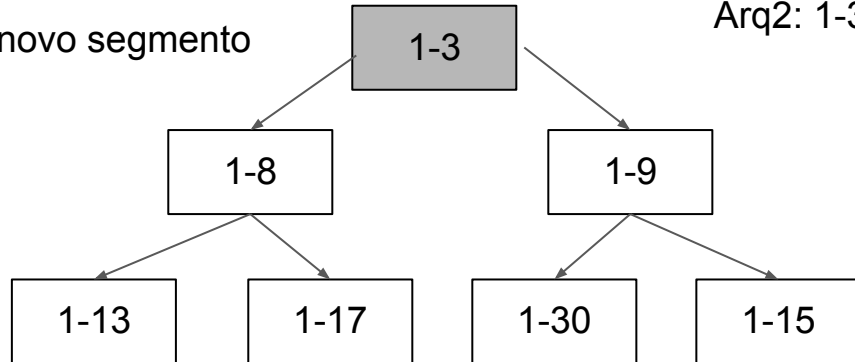
organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2: 1-3;

novo segmento



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

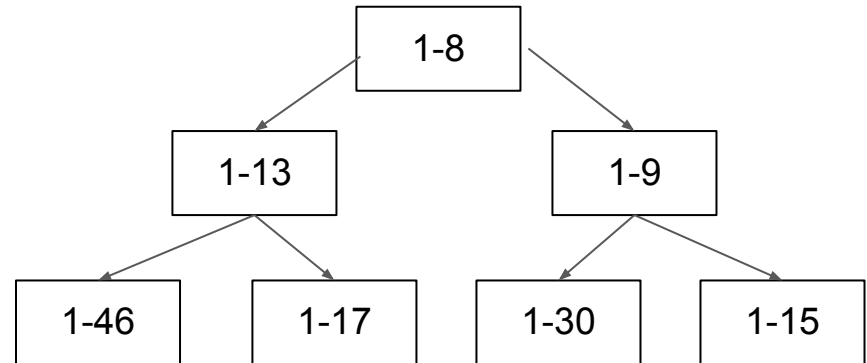
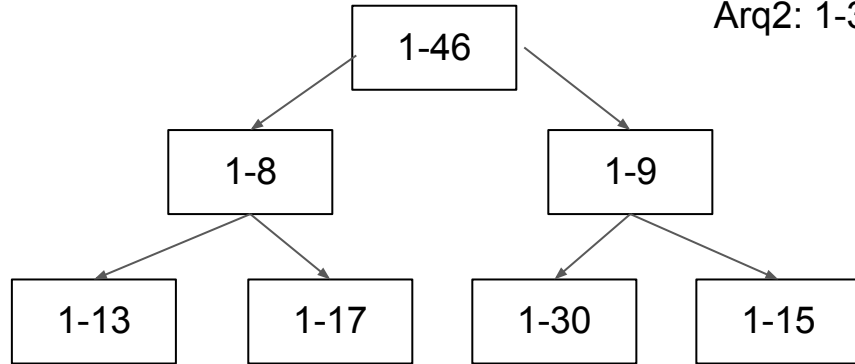
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2: 1-3;



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

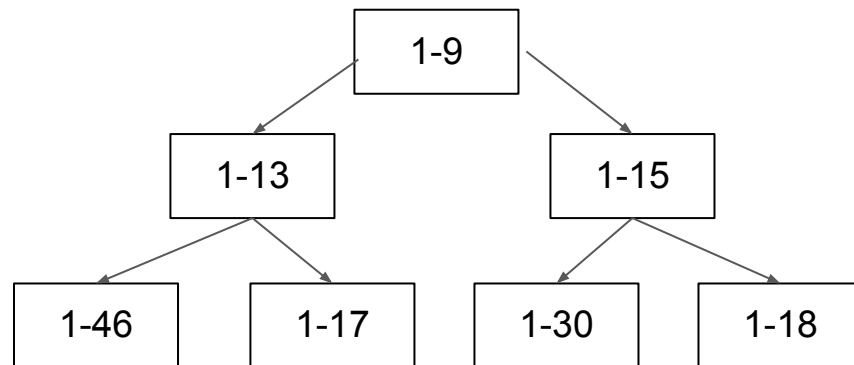
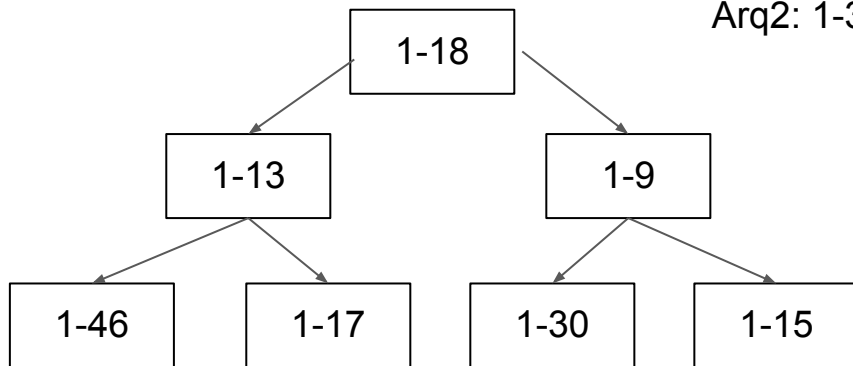
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2: 1-3; 1-8;



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

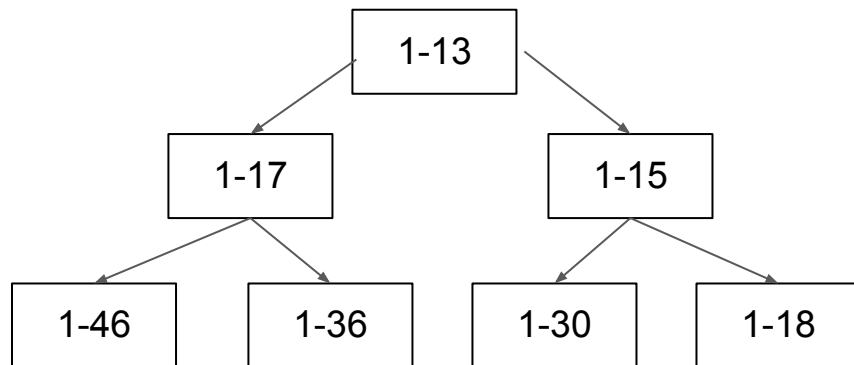
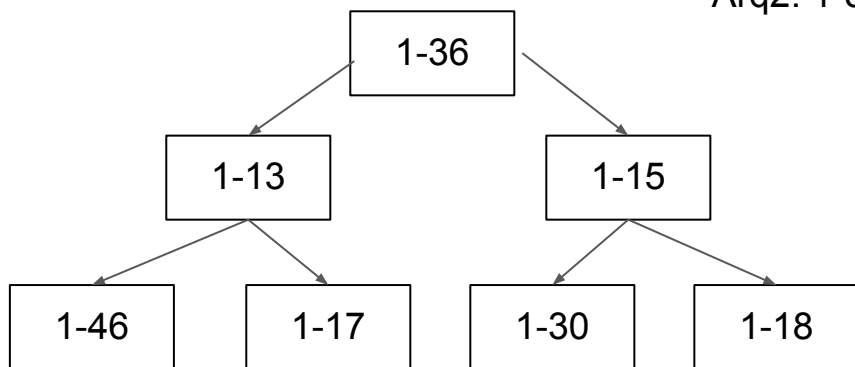
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2: 1-3; 1-8; 1-9;



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

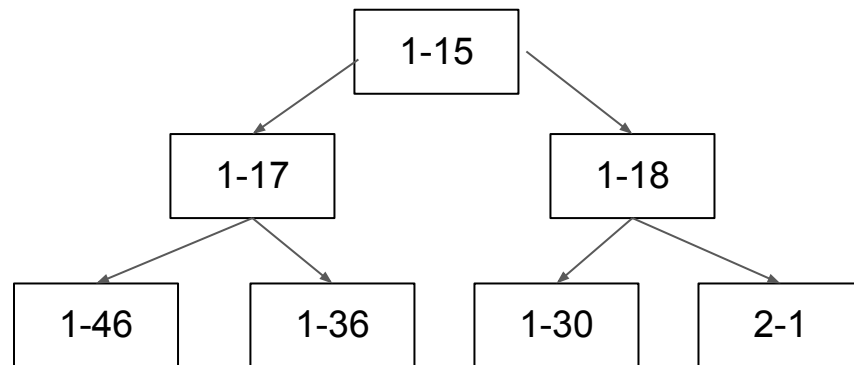
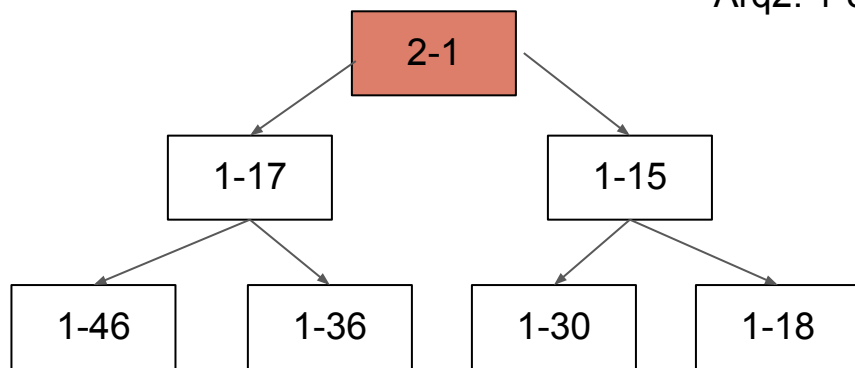
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2: 1-3; 1-8; 1-9; 1-13;



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

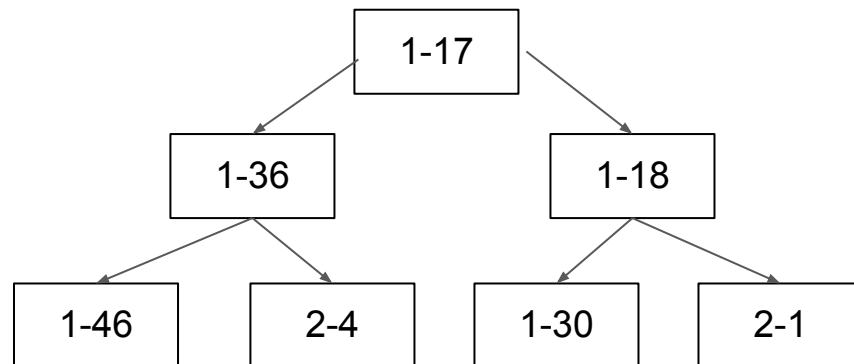
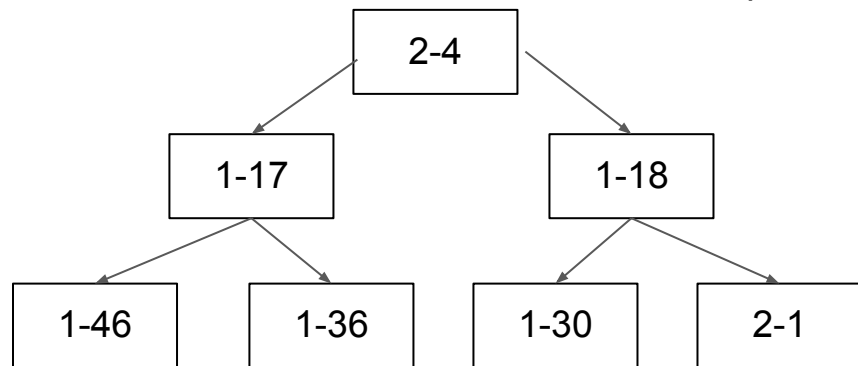
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2: 1-3; 1-8; 1-9; 1-13;1-15;



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

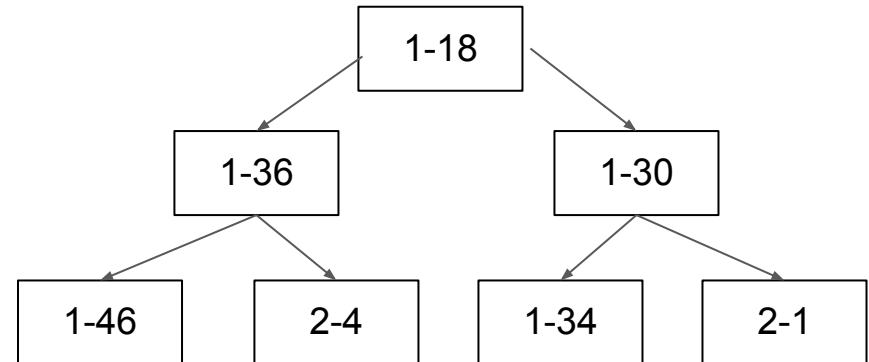
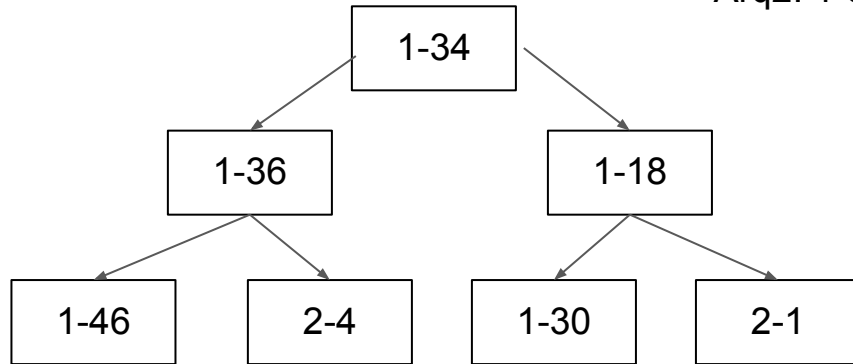
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2: 1-3; 1-8; 1-9; 1-13; 1-15; 1-17;



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

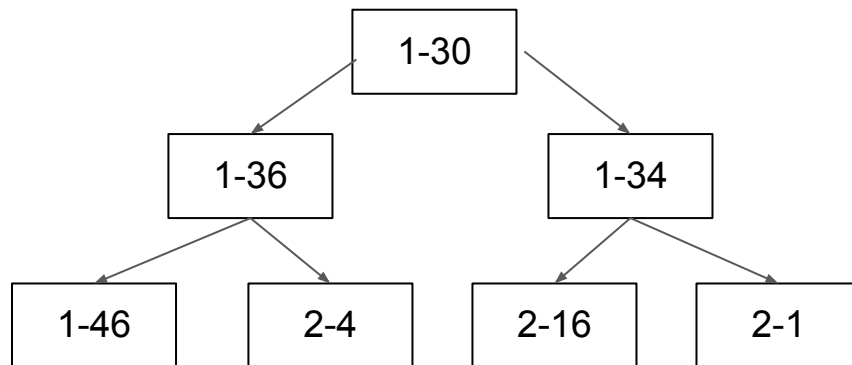
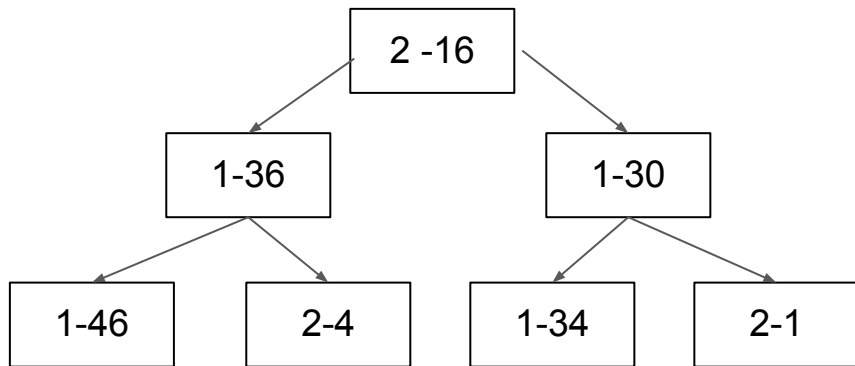
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2: 1-3; 1-8; 1-9; 1-13; 1-15; 1-17; 1-18;



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

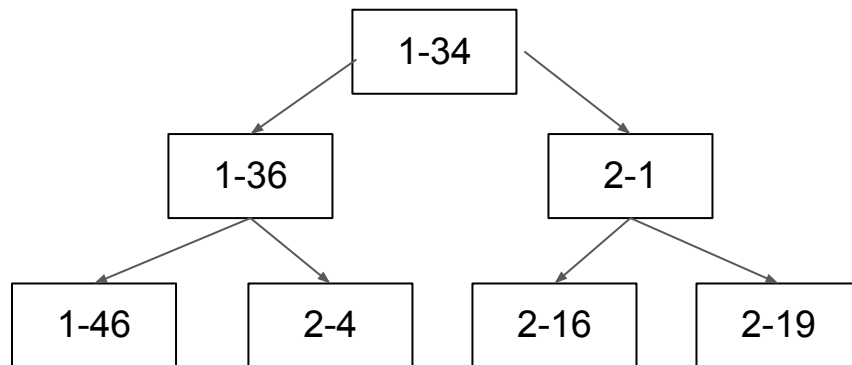
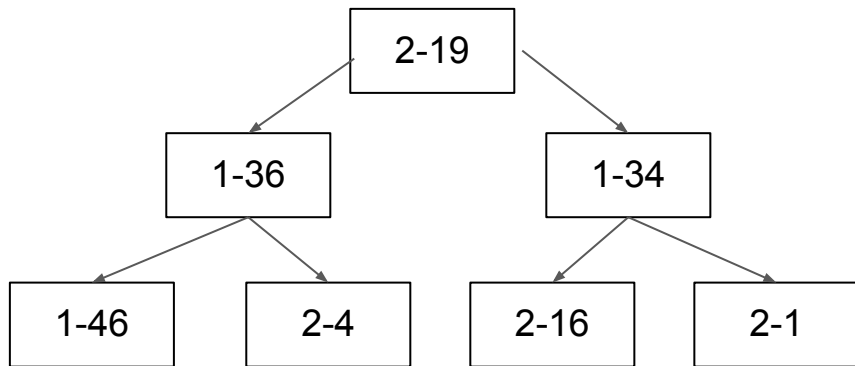
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2: 1-3; 1-8; 1-9; 1-13; 1-15; 1-17; 1-18; 1-30;



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

coloca o menor no arquivo de saída

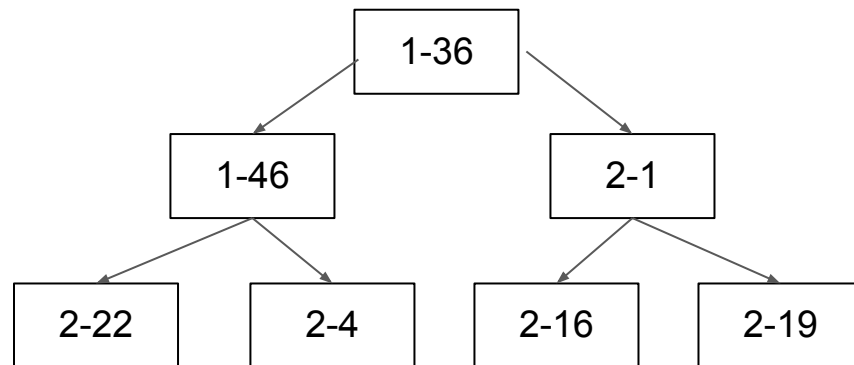
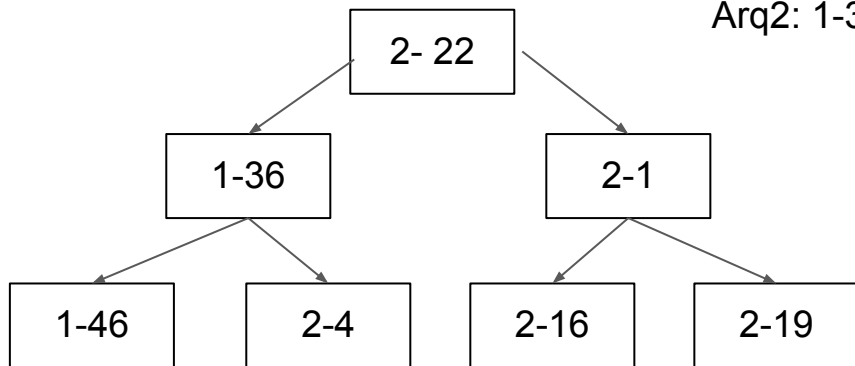
e insere próximo registro na raiz

organiza a heap

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2: 1-3; 1-8; 1-9; 1-13;1-15; 1-17; 1-18;1-30;1-34;



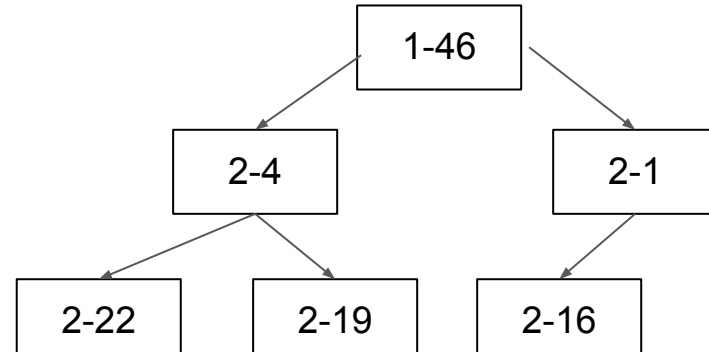
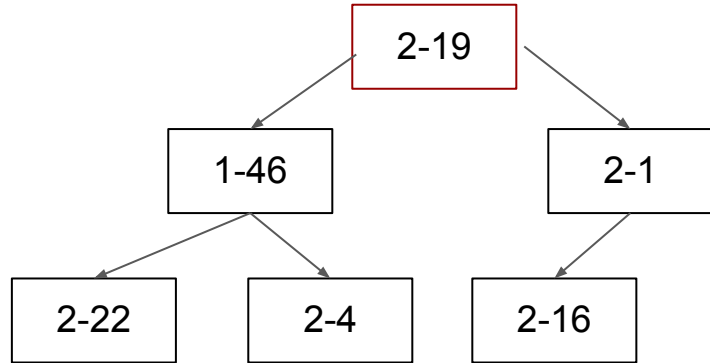
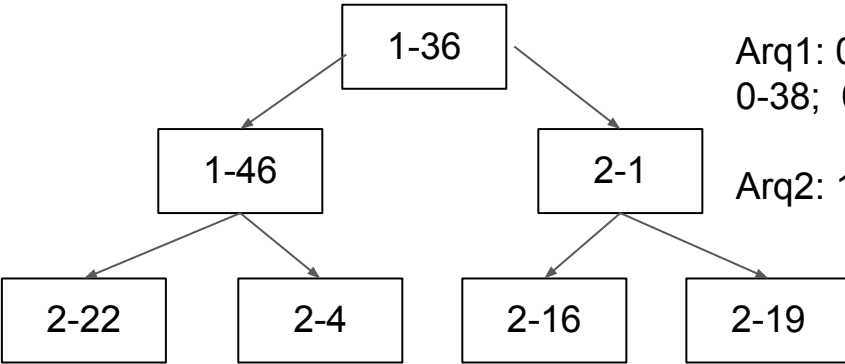
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

esvaziamento da heap

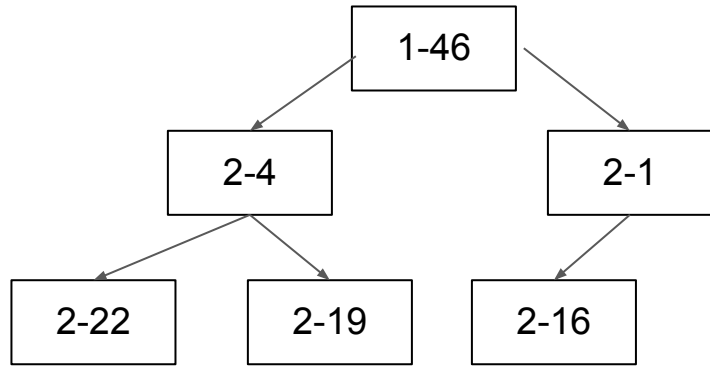
saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2: 1-3; 1-8; 1-9; 1-13; 1-15; 1-17; 1-18; 1-30; 1-34; 1-36;



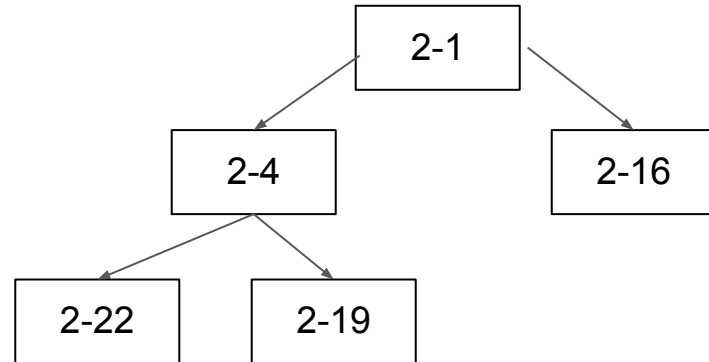
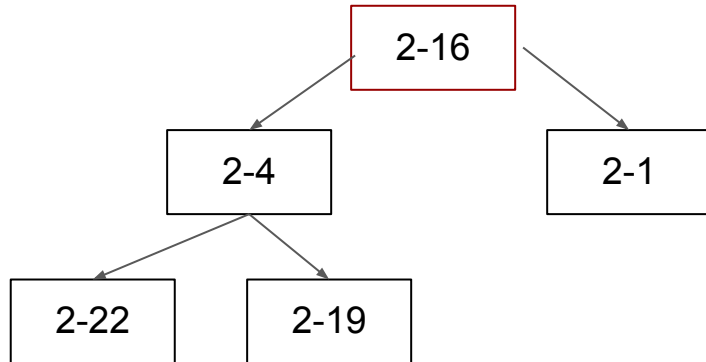
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22



saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49;

Arq2: 1-3; 1-8; 1-9; 1-13;1-15; 1-17; 1-18;1-30;1-34; 1-36; 1-46;



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	28	10	40	35	7	12	6	21	11	29	27	9	38	8	49	3	15	13	30	17	46	18	36	1	4	34	16	19	22

saídas:

Arq1: 0-5; 0-6; 0-7; 0-10; 0-11; 0-12; 0-21; 0-27; 0-28; 0-29; 0-35;
0-38; 0-40; 0-49; 2-1; 2-4; 2-16; 2-19; 2-22;

Arq2: 1-3; 1-8; 1-9; 1-13;1-15; 1-17; 1-18;1-30;1-34; 1-36; 1-46;

Agora passamos para as intercalações!